

Assignment -1 CSL7490

Instructor: Susil Kumar Mohanty

Due Date: 20/02/2026 11:59 PM SGT

Objective: Design a blockchain-based petroleum supply chain ledger on top of a P2P network

Overview: The network consists of **Seed Nodes** and **Peer Nodes**.

- **Registration:** Every new peer registers with at least $\lfloor n/2 \rfloor + 1$ seeds.
- **P2P Connectivity:** Peers obtain a Peer List (PL) from seeds, take a union of all lists, and select at least 4 **distinct peer nodes** to establish TCP connections. The resulting network must be a connected graph.

Cryptographic Identity and Address Generation

Each node must establish a unique identity before participating.

- **Key Generation:** Use the **secp256k1** elliptic curve ($y^2 \equiv x^3 + 7 \pmod{p}$). Generate a **Private/Secret Key**.
- **Address Derivation:** The **Wallet Address** is created by taking the **SHA-256 hash** of the Public Key and using the **last 16 bits** in a 4-digit hexadecimal format (e.g., 0x9e1c).
- **Generate a random number and use SHA256 hash algorithm to generate Secret Key (sk)** (256-bit integer) and derive a **Public Key (pk)** using the **Double-and-Add algorithm** ($pk = sk \cdot G$).

Transaction Architecture and Lifecycle

Transactions represent events such as supply chain updates (e.g., "100 barrels delivered") or financial transfers (e.g., Alice sending \$10 to Bob).

- **Transaction Object:** Each transaction must contain:
 - **TxID:** A **SHA-256 hash** of the transaction's contents to ensure integrity (the Avalanche Effect).
 - **SenderAddress & ReceiverAddress:** The 16-bit hexadecimal identifiers of the participants.
 - **Data:** The specific details of the event (e.g., "Alice pays Bob \$10").
- **Digital Signatures:** To ensure non-repudiation, the sender must sign the transaction using **ECDSA**.
 - The sender uses their **sk** to create a signature pair (**r, s**).
 - **Verification:** Receiving nodes use the sender's **pk** to verify the signature. A valid signature proves the sender authorised the transaction and the data is untampered.

Gossip Protocol and Network Communication

Transactions and blocks are propagated across the network through a Gossip protocol.

- **Gossip Format:** Peers generate messages in the format:
`<self.timestamp>:<self.IP>:<self.Msg#>`.
- **Relaying:** Nodes maintain a **Message List (ML)** containing hashes of received messages. When a node receives a new transaction, it checks the ML; if unique, it makes an entry, validates the signature, and forwards the message to all adjacent peers except the sender.
- **Liveness:** Nodes exchange **Liveness Requests** every 13 seconds. If 3 consecutive requests fail, the peer reports the '**Dead**' node to its connected seeds and closes the connection.

Reporting the node as 'Dead'

When 3 consecutive liveness requests do not receive a reply, the peer sends a message of the following format to all the seeds it is connected to:

`Dead Node:<DeadNode.IP>:<DeadNode.Port>:<self.timestamp>:<self.IP>`

Block Mining

We will simulate PoW mining in the following way. When a miner receives a block at say time t , it will generate an exponential random variable, say τ . Note that τ depends on the hash power assigned to the miner. The miner will wait up to time $t + \tau$. If it has not yet received another block by then, it will generate a block and broadcast it. If, on the other hand, it receives a block (which creates a longer chain than its current chain) before $t + \tau$, it will generate a new exponential random variable, and the process repeats.

If *interarrival time* represents the average block interarrival in the entire network, then you can use the following steps to generate the exponential random variable described above.

- Overall block generation rate
 $meanT_k = 1.0/interarrivaltime$
- Scale the individual miners' lambda parameter based on the percentage of hash power it has.
 $lambda = nodeHashPower * meanT_k / 100.0$
- Appropriately scale the exponential waiting time
 $T_k := time.Duration(rand.ExpFloat64()/lambda)$

A newly joined node, say N , should sync with the blockchain's current height in the network. N will receive the recent block B_k , where k is the height, from the network immediately after joining the network. N won't push the B_k onto the chain; it will be in the pending queue. N will request blocks from B_0 (genesis block) to B_{k-1} from the network and put each on the chain, post-validation. All the blocks received after B_k will enter the pending queue. After the node N syncs till $k - 1$, it starts processing the pending queue. For simplicity, the limit on the size of the pending queue is unrestricted. An honest node will begin creating and mining the block once the pending queue is empty.

While a node is mining, two events can occur: a) The node itself comes up with the solution, in our case, the timer expires, or b) The node receives the block from another node in the network, and this block creates a longer chain than it had before receiving the block.

- a) After the node creates the block, it should follow the following steps :

- Store the block in the database.
- Broadcast the block to neighbors.

b) After receiving a block, the node should insert the block into the pending queue and abort the mining process if it's running.

Node validates the blocks in the pending queue one at a time. Validation consists of the following steps :

- Validate the received block (check if the hash in the block is the hash of some other existing block; timestamp was generated within 1 hour (plus or minus) of the current time.) Reject the block if it fails the validation test. Do the following.
- Store it in the database (you are free to use any database that you are comfortable with).
- Broadcast the block to neighbors in the P2P network.
- If the block creates a chain longer than the longest chain currently and the pending queue is empty, reset the timer for the next block waiting time.

Note that the node should always mine on the **longest chain** available locally.

Experimental Tasks and Analytical Questions

1: The Avalanche Effect

Task: Take a standard transaction string and compute its SHA-256 hash. Change one character in the string and recompute the hash.

Question: What percentage of the bits changed in the second hash? How does this property support the **tamper-resistance** of the blockchain?

2: Merkle Proof Efficiency

Task: Create a Merkle Tree for a block containing 8 transactions. Provide the **Merkle Proof** for the 4th transaction.

Question: Why is providing a Merkle Proof more efficient for a **Light Node** than downloading the entire block body?

3: Double-Spending Simulation

Task: Simulate an event where an Upstream Producer attempts to record the same 100 barrels of oil as delivered to two different refineries simultaneously.

Question: How do the **Public Trusted Bulletin Board** (the ledger) and the consensus mechanism prevent this "double spending" of assets?

4: Mining Power and Forks

Task: Simulate a network where one node has 51% of the total hash power.

Question: Drawing on the **Blockchain Trilemma**, what happens to the network's **Security** and **Decentralisation** if a single entity controls the majority of the computing power?

5: Bit Commitment in Supply Chain

Task: Implement a **Commitment Scheme** where a producer commits to a delivery volume m with a nonce r ($C=H(m \parallel r)$) during the "Exploration" phase but only reveals it during "Refining".

Question: How do the **Hiding** and **Binding** properties of this scheme resolve coordination disputes between mutually distrustful parties?

6. Target Variable and Leading Zeros Task

Task : we simulate Proof of Work (PoW). In a live blockchain, nodes must find a hash that is less than a specific **target value**, which is often represented by a required number of **leading zeros**. Analyze the relationship between bitwise difficulty and our simulation parameters.

Question: In a standard PoW implementation, for a given **value of the target variable**, **how many bits of leading zeros must be added** to ensure the hash meets the difficulty requirement? How would an increase in the required bits of leading zeros mathematically affect the **lambda parameter** (λ) and the resulting **waiting time** (τ) in our simulation?

Task 7: Stochastic Analysis of Mining Probability

Task: Using graph plotting tools such as **MATLAB, Python, or LaTeX**, analyze the statistical behavior of the simulated Proof of Work process based on the variables λ and T_k .

1. **Waiting Time Distribution:** For a node with a fixed percentage of **hash power**, collect the values of the **waiting time** (T_k) generated over 100 mining cycles. Plot these values as a histogram or a probability density function (PDF).

2. **Lambda vs. Hash Power:** Generate a plot showing the relationship between a node's **individual lambda** (λ) and its assigned **hash power percentage** (from 1% to 100%), given a constant **overall block generation rate** ($\text{mean}T_k$).

Question: Based on your plot, how does the **exponential random variable** behave as a node's hash power increases? Specifically, how does a higher λ value, calculated as $\lambda=(\text{nodeHashPower} \times \text{mean}T_k)/100.0$, impact the average **waiting time** (T_k)?

You can use the following platforms.

1. OMNeT++
2. VS Code
3. OpenSSL Library
4. Socket Programming
5. Graph plotting (MATLAB, Python, Latex)

Submission Instructions:

1. The assignment should be done in a group consisting of a maximum of **THREE** students.

2. The code should follow programming etiquette with appropriate comments.
3. Add a **README** file that includes a description of the code and gives detailed steps to compile and run the code.
4. Zip all your code and the report as a single file with the name **rollno1-rollno2.tar.gz** and upload it to Google Classroom. Only one group member should upload the file.
5. You should be able to **demonstrate** the code.